

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 3: Desarrollo de aplicaciones web distribuidas

Tema 2: Programación de servicios web en entornos distribuidos

Componentes software para el acceso a servicios distribuidos

Programación de diferentes tipos de acceso a servicios

Herramientas para la programación de servicios web

Componentes software para el acceso a servicios distribuidos

1 Definición de servicios

Un **servicio web** es una funcionalidad accesible mediante una red (habitualmente internet) que sigue ciertos protocolos para ser descubierta, consumida y reutilizada por otras aplicaciones.

Se basa en tres pilares:

- **Interfaz pública:** describe qué operaciones ofrece (por ejemplo, GET /productos).
- **Protocolo de comunicación:** generalmente HTTP o HTTPS.
- **Formato de datos:** XML o JSON son los más comunes.

Ejemplo:

Un servicio REST que devuelve productos desde un servidor:

GET <https://api.mitienda.com/productos>

2 Generación automática de servicios

La **generación automática** implica crear el servicio a partir de definiciones ya estructuradas. En entornos más complejos (como con SOAP o WSDL), se pueden generar automáticamente los ficheros necesarios para el cliente y el servidor.

En entornos modernos (como con **OpenAPI/Swagger**), se puede:

- Generar la documentación de la API automáticamente.
- Generar código cliente y servidor con herramientas como Swagger Codegen.

Ejemplo con Swagger:

Definición del servicio en un archivo .yaml, que genera automáticamente el punto de acceso, la documentación y los esquemas de datos.

Programación de diferentes tipos de acceso a servicios

1 Servicios basados en publicación/suscripción.

- El proveedor publica un evento.
- Los suscriptores lo reciben si están registrados.

Ejemplo:

Un servidor publica cambios en tiempo real (ej. nuevos mensajes), y los navegadores suscritos lo reciben mediante WebSockets o tecnologías como Pusher o Firebase.

2 Servicios basados en repositorios

Son servicios que centralizan información (por ejemplo, una API de documentos o de imágenes).

Ejemplo:

Un servicio REST que permite GET /documentos o POST /documentos para añadir contenido a un repositorio.

3 Servicios accesibles desde agentes de usuario

Aplicaciones cliente (navegadores o apps móviles) que consumen servicios vía HTTP.

Ejemplo en JS:

```
fetch('https://api.openweathermap.org/data/2.5/weather?q=A Coruña')  
  .then(response => response.json())  
  .then(data => console.log(data));
```

4 Proveedores y consumidores de servicios en entorno servidor

- **Proveedor:** servidor que expone una API.
- **Consumidor:** aplicación (cliente) que usa esa API para integrar funcionalidades.

Ejemplo en PHP (consumidor):

```
$json = file_get_contents('https://api.thecatapi.com/v1/images/search');  
$data = json_decode($json, true);  
echo '<img src="' . $data[0]['url'] . '>';
```

Herramientas para la programación de servicios web

1 Comparativa

Herramienta	Lenguaje	Ideal para	Pros	Contras	
cURL	cURL	PHP	Peticiones HTTP simples	Versátil y directo	Poco visual
Guzzle	Guzzle	PHP	Clientes HTTP avanzados	Fácil integración, manejo de errores	Requiere instalación vía Composer
Axios	Axios	JS	Frontend moderno	Promesas, JSON por defecto	No funciona en PHP
Postman	Postman	N/A	Testeo de APIs	Interfaz gráfica, pruebas automatizadas	Solo para desarrollo, no en producción

2 Bibliotecas y entornos integrados (frameworks) de uso común.

Entorno	Descripción	Uso
Laravel (PHP)	Framework completo que incluye generación de servicios	Crear APIs RESTful
Symfony (PHP)	Más estructurado y extensible, ideal para grandes proyectos	Servicios organizados y seguros
Slim (PHP)	Micro-framework ligero para crear servicios REST	Ideal para APIs rápidas
Express (Node.js)	Muy usado para APIs JS	Muy flexible para servicios JSON

Ejercicio práctico

Enunciado:

Crea un servicio REST simulado con PHP sin base de datos.

Debe permitir obtener una lista de productos en formato JSON desde productos.php.

Glosario de términos clave

Término	Descripción
REST	Estilo de arquitectura basada en recursos, usa HTTP estándar
SOAP	Protocolo para servicios web más formal y estructurado
API	Interfaz que expone funcionalidades a otros programas
Cliente	Aplicación que consume una API
Proveedor	Aplicación que ofrece una API
JSON	Formato ligero de intercambio de datos
XML	Formato estructurado más robusto, pero más pesado
Swagger	Herramienta para documentar y generar APIs REST
Fetch	Función de JS para hacer peticiones HTTP
Webhook	Llamada automática a un servicio externo ante un evento